JAVA PROGRAMMING LABORATORY

INDUSTRY PROBLEM-SOLVING TASK

Java AWT, Applets, Event Handling and Layout Managers

Name of the Student	Jeyakannan	**Register No.**	192525376
Programme / Section	CO4-AT1	**Date**	23-08-2026
Course Code	CSA09	**Course**	Programming in Java
Faculty	Dr.Sivasankar	**Duration**	Lab Session

INDUSTRY TASK 1: Employee Performance & Payroll Dashboard

Problem Statement: A software company wants to develop a simple Employee Performance and Payroll Management System using Java AWT. The application should allow the HR team to enter employee details, process salary information, and generate a structured payroll report.

Functional Requirements

- Develop an AWT-based graphical user interface.
- Accept Employee ID, Employee Name, Department, Designation, Basic Salary, Working Days, and Performance Rating.
- Use suitable AWT components such as Label, TextField, Choice, Checkbox/CheckboxGroup, Button, TextArea, and List.
- Calculate Gross Salary, Performance Bonus, and Net Salary using suitable business rules defined by the student.
- Display the complete employee payroll report in a List or TextArea when Generate Report is clicked.
- Use GridBagLayout or BorderLayout for arranging the interface.
- Implement button events using ActionListener.
- Provide Submit/Generate Report and Reset functionality.
- Validate important inputs such as empty fields and numeric salary values.

Industry Context: The application simulates an HR payroll module used to capture employee information, perform basic payroll processing, and present a report for HR verification.

Answer :

Java Code :

```
import java.awt.*;

import java.awt.event.*;


public class EmployeePayrollSystem extends Frame implements ActionListener {


    TextField idField, nameField, salaryField, daysField;


    Choice departmentChoice, designationChoice;
```

```
CheckboxGroup ratingGroup;
Checkbox rating1, rating2, rating3, rating4, rating5;

Button generateButton, resetButton;

TextArea reportArea;

List employeeList;

public EmployeePayrollSystem() {

    setTitle("Employee Performance and Payroll Management System");

    setSize(900, 650);

    setLayout(new BorderLayout(10, 10));

    // Title
    Label title = new Label(
        "EMPLOYEE PERFORMANCE AND PAYROLL MANAGEMENT SYSTEM",
        Label.CENTER
    );

    title.setFont(new Font("Arial", Font.BOLD, 20));

    add(title, BorderLayout.NORTH);
```

```
// Main Form
Panel formPanel = new Panel();

formPanel.setLayout(new GridBagLayout());

GridBagConstraints gbc = new GridBagConstraints();

gbc.insets = new Insets(8, 8, 8, 8);

gbc.fill = GridBagConstraints.HORIZONTAL;


// Employee ID
gbc.gridx = 0;
gbc.gridy = 0;

formPanel.add(new Label("Employee ID:"), gbc);

idField = new TextField(20);

gbc.gridx = 1;

formPanel.add(idField, gbc);


// Employee Name
gbc.gridx = 0;
gbc.gridy = 1;
```

```
formPanel.add(new Label("Employee Name:"), gbc);

nameField = new TextField(20);

gbc.gridx = 1;

formPanel.add(nameField, gbc);


// Department
gbc.gridx = 0;
gbc.gridy = 2;

formPanel.add(new Label("Department:"), gbc);

departmentChoice = new Choice();

departmentChoice.add("IT");
departmentChoice.add("HR");
departmentChoice.add("Finance");
departmentChoice.add("Marketing");
departmentChoice.add("Operations");

gbc.gridx = 1;

formPanel.add(departmentChoice, gbc);


// Designation
```

```
gbc.gridx = 0;
gbc.gridy = 3;

formPanel.add(new Label("Designation:"), gbc);

designationChoice = new Choice();

designationChoice.add("Software Engineer");
designationChoice.add("Team Leader");
designationChoice.add("Project Manager");
designationChoice.add("HR Executive");
designationChoice.add("Data Analyst");

gbc.gridx = 1;

formPanel.add(designationChoice, gbc);



// Basic Salary
gbc.gridx = 0;
gbc.gridy = 4;

formPanel.add(new Label("Basic Salary:"), gbc);

salaryField = new TextField(20);

gbc.gridx = 1;

formPanel.add(salaryField, gbc);
```

```
// Working Days
gbc.gridx = 0;
gbc.gridy = 5;

formPanel.add(new Label("Working Days:"), gbc);

daysField = new TextField(20);

gbc.gridx = 1;

formPanel.add(daysField, gbc);



// Performance Rating
gbc.gridx = 0;
gbc.gridy = 6;

formPanel.add(new Label("Performance Rating:"), gbc);



ratingGroup = new CheckboxGroup();

rating1 = new Checkbox("1", ratingGroup, false);
rating2 = new Checkbox("2", ratingGroup, false);
rating3 = new Checkbox("3", ratingGroup, true);
rating4 = new Checkbox("4", ratingGroup, false);
rating5 = new Checkbox("5", ratingGroup, false);
```

```
Panel ratingPanel = new Panel();

ratingPanel.add(rating1);
ratingPanel.add(rating2);
ratingPanel.add(rating3);
ratingPanel.add(rating4);
ratingPanel.add(rating5);


gbc.gridx = 1;

formPanel.add(ratingPanel, gbc);


// Buttons
generateButton = new Button("Generate Report");

resetButton = new Button("Reset");


Panel buttonPanel = new Panel();

buttonPanel.add(generateButton);
buttonPanel.add(resetButton);


gbc.gridx = 0;
```

```
gbc.gridy = 7;

gbc.gridwidth = 2;


formPanel.add(buttonPanel, gbc);



add(formPanel, BorderLayout.CENTER);



// Report Panel

Panel reportPanel = new Panel();


reportPanel.setLayout(new BorderLayout(5, 5));



Label reportLabel = new Label(

    "PAYROLL REPORT",

    Label.CENTER

);


reportArea = new TextArea();


reportArea.setEditable(false);



reportPanel.add(reportLabel, BorderLayout.NORTH);


reportPanel.add(reportArea, BorderLayout.CENTER);
```

```
// Employee List
employeeList = new List(6);


reportPanel.add(employeeList, BorderLayout.SOUTH);



add(reportPanel, BorderLayout.EAST);



// Button Events
generateButton.addActionListener(this);


resetButton.addActionListener(this);



// Window Closing
addWindowListener(
    new WindowAdapter() {

        public void windowClosing(WindowEvent e) {

            System.exit(0);

        }
    }
);
```

```
    setLocationRelativeTo(null);

    setVisible(true);
}

// Button Event Handling
public void actionPerformed(ActionEvent e) {

    if (e.getSource() == generateButton) {

        generateReport();

    }

    if (e.getSource() == resetButton) {

        resetForm();

    }
}

// Generate Payroll Report
public void generateReport() {

    String employeeID = idField.getText().trim();
```

```
String employeeName = nameField.getText().trim();


String department =

   departmentChoice.getSelectedItem();


String designation =

   designationChoice.getSelectedItem();



// Check Empty Fields

if (employeeID.isEmpty()

        || employeeName.isEmpty()

        || salaryField.getText().isEmpty()

        || daysField.getText().isEmpty()) {


   showMessage("Please fill all fields.");


   return;

}



double basicSalary;


int workingDays;



// Convert Salary and Days

try {
```

```
    basicSalary =
        Double.parseDouble(
            salaryField.getText()
        );

    workingDays =
        Integer.parseInt(
            daysField.getText()
        );

}
catch (NumberFormatException ex) {

    showMessage(
        "Enter valid numeric values for salary and working days."
    );

    return;
}



// Validate Salary
if (basicSalary <= 0) {

    showMessage(
        "Salary must be greater than zero."
    );

    return;
```

```
}

// Validate Working Days
if (workingDays < 1 || workingDays > 31) {

    showMessage(
        "Working days must be between 1 and 31."
    );

    return;
}

// Get Rating
Checkbox selectedRating =
    ratingGroup.getSelectedCheckbox();

if (selectedRating == null) {

    showMessage(
        "Please select performance rating."
    );

    return;
}
```

```
int rating =
    Integer.parseInt(
        selectedRating.getLabel()
    );


// Salary Calculation

// Allowance = 20% of Basic Salary
double allowance =
    basicSalary * 0.20;


// Gross Salary
double grossSalary =
    basicSalary + allowance;


// Performance Bonus
double bonusPercentage;


if (rating == 5) {

    bonusPercentage = 0.20;

}
else if (rating == 4) {
```

```
    bonusPercentage = 0.15;

}
else if (rating == 3) {

    bonusPercentage = 0.10;

}
else if (rating == 2) {

    bonusPercentage = 0.05;

}
else {

    bonusPercentage = 0.0;

}


double performanceBonus =
    basicSalary * bonusPercentage;


// Net Salary
double netSalary =
    grossSalary + performanceBonus;
```

```
// Create Report
String report =
    "======================================\n"
    + "          PAYROLL REPORT\n"
    + "======================================\n"
    + "Employee ID        : " + employeeID + "\n"
    + "Employee Name      : " + employeeName + "\n"
    + "Department         : " + department + "\n"
    + "Designation        : " + designation + "\n"
    + "Working Days       : " + workingDays + "\n"
    + "Performance Rating : " + rating + "\n"
    + "--------------------------------------\n"
    + String.format(
        "Basic Salary       : %.2f%n",
        basicSalary
    )
    + String.format(
        "Allowance          : %.2f%n",
        allowance
    )
    + String.format(
        "Gross Salary       : %.2f%n",
        grossSalary
    )
    + String.format(
        "Performance Bonus  : %.2f%n",
        performanceBonus
    )
    + "--------------------------------------\n"
```

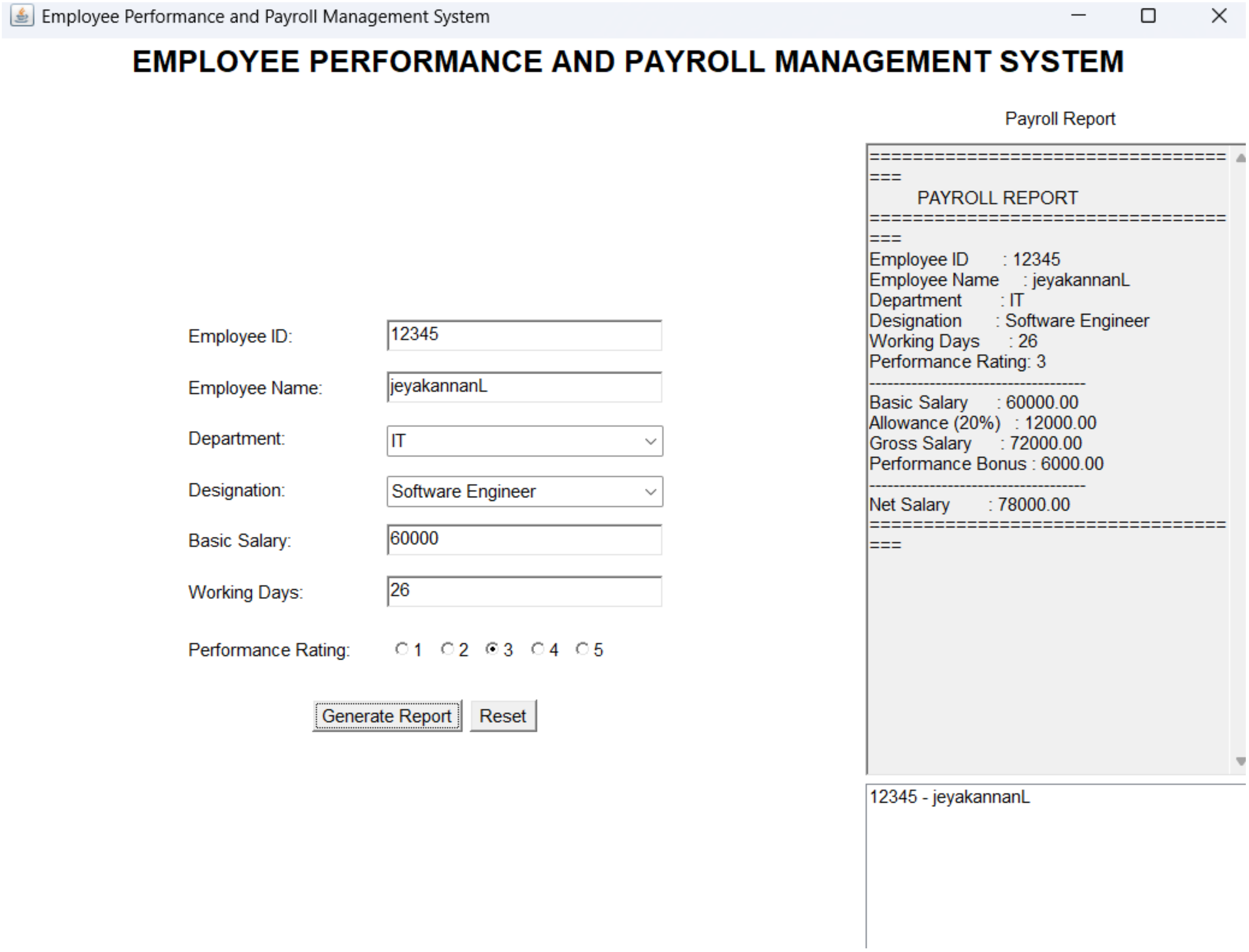
```
        + String.format(
            "Net Salary        : %.2f%n",
            netSalary
        )
        + "====================================";


    // Display Report
    reportArea.setText(report);


    // Add Employee to List
    employeeList.add(
        employeeID + " - " + employeeName
    );
}


// Reset Form
public void resetForm() {

    idField.setText("");

    nameField.setText("");

    salaryField.setText("");

    daysField.setText("");
```

```
    departmentChoice.select(0);

    designationChoice.select(0);

    ratingGroup.setSelectedCheckbox(
        rating3
    );

    reportArea.setText("");
}

// Message Box
public void showMessage(String message) {

    Dialog dialog =
        new Dialog(
            this,
            "Message",
            true
        );

    dialog.setLayout(
        new BorderLayout()
    );
```

```
Label messageLabel =
    new Label(
        message,
        Label.CENTER
    );


Button okButton =
    new Button("OK");


dialog.add(
    messageLabel,
    BorderLayout.CENTER
);


dialog.add(
    okButton,
    BorderLayout.SOUTH
);


okButton.addActionListener(
    new ActionListener() {

        public void actionPerformed(
```

```
                ActionEvent e

            ) {

                dialog.dispose();

            }

        }

    );

    dialog.setSize(400, 150);

    dialog.setLocationRelativeTo(this);

    dialog.setVisible(true);

}

// Main Method

public static void main(String[] args) {

    new EmployeePayrollSystem();

}

}
```

Output Screen shot :



Output Report :

===================================

PAYROLL REPORT

===================================

Employee ID : 12345

Employee Name : jeyakannanL

Department : IT

Designation : Software Engineer

Working Days : 26

Performance Rating: 3

Basic Salary : 60000.00

Allowance (20%) : 12000.00

Gross Salary : 72000.00

Performance Bonus : 6000.00

Net Salary : 78000.00

INDUSTRY TASK 2: Online Food Ordering and Billing System

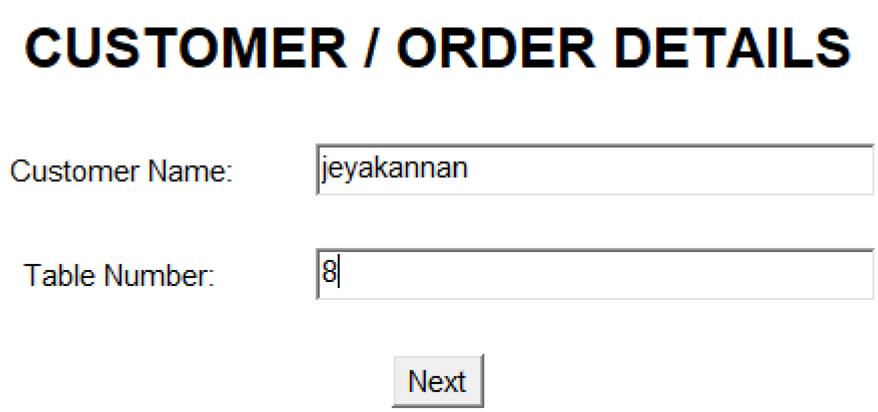
Problem Statement: A restaurant wants to develop a simple Food Ordering and Billing Application for its counter staff. The application should allow customers to select food items, specify quantities, and generate a final bill.

Functional Requirements

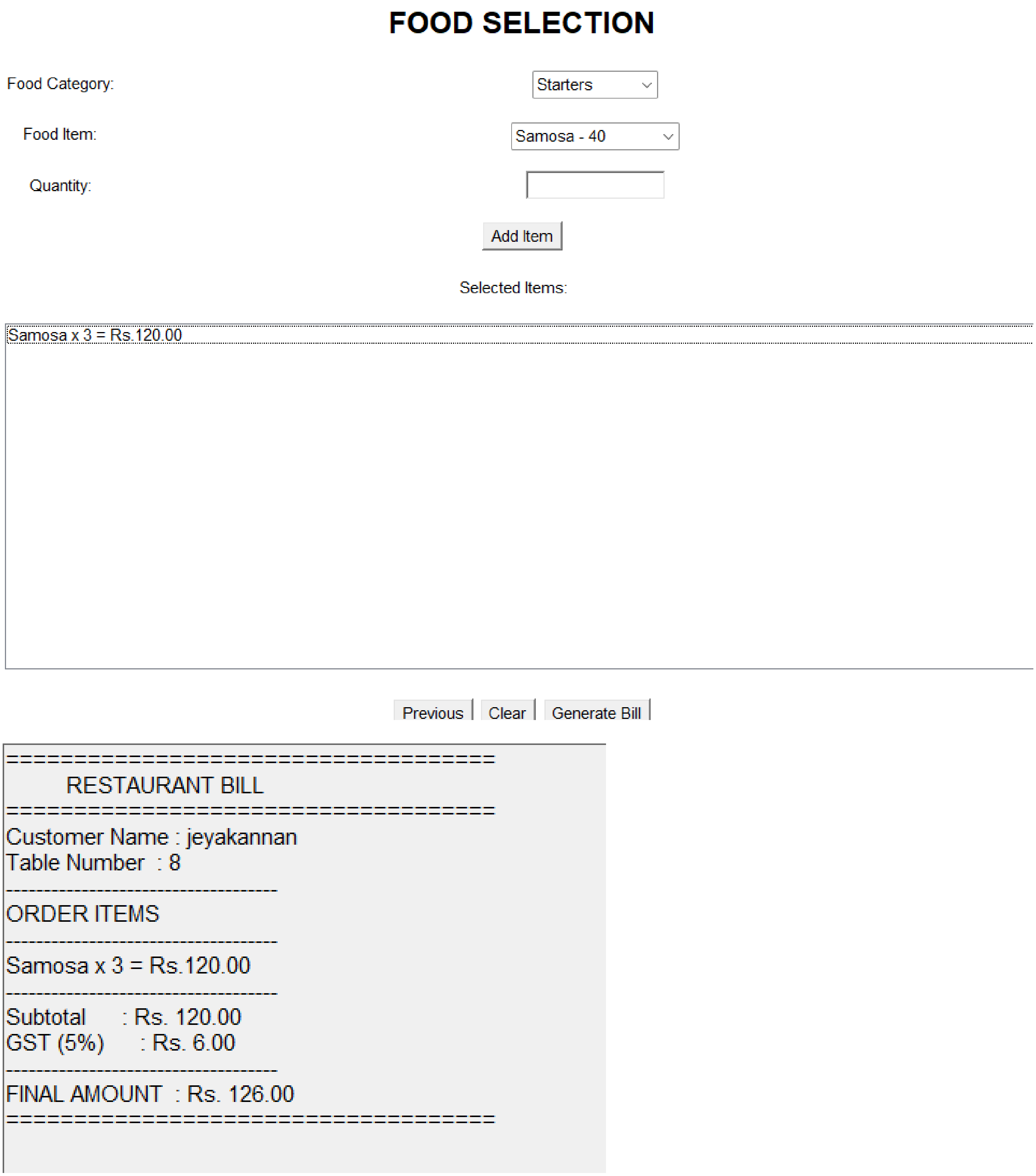
- Develop the application using Java AWT.
- Create fields for Customer Name and Table Number.
- Provide Food Category, Food Item, and Quantity selection using appropriate AWT components.
- Provide Add Item, Generate Bill, Clear, Next, and Previous buttons.
- Display selected food items in a List or TextArea.
- Calculate item-wise cost, Subtotal, GST, and Final Bill Amount.
- Implement button click events using ActionListener.
- Use CardLayout to create separate screens for Customer/Order Details, Food Selection, and Bill/Order Summary.
- Validate quantity and ensure that an item is selected before adding it to the order.

Industry Context: The application represents a simplified restaurant POS (Point of Sale) system used for taking customer orders, processing prices, and generating a customer bill.

Output screen shots :



2 . Food selection :



Evaluation Rubric

Criterion	Weightage	Marks / Remarks
Problem Understanding	15%	
GUI Components and Layout	20%	
Event Handling	20%	

Functional Correctness	25%	
Input Validation and User Experience	10%	
Program Explanation / Viva	10%	